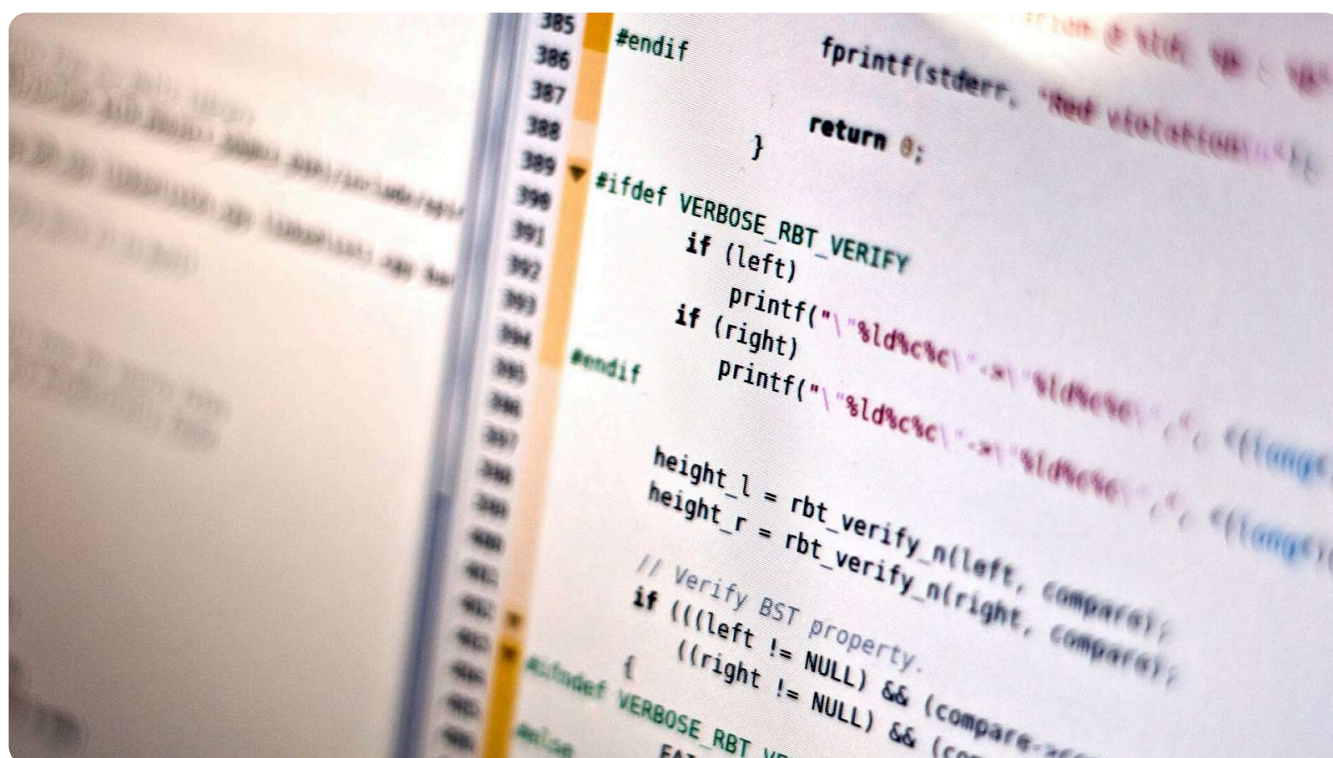


C++11 Lambda Expression 語法教學與範例

February 19, 2015 · G. T. Wang



► 目錄

C++11 標準中所新增的 lambda expression 語法，可以讓函數的定義與使用更加有彈性，程式碼看起來也更簡潔。

C++11 的標準中加入了一個新的 lambda expression 語法，如果您有一陣子沒有注意最新的 C++ 標準，看到這樣的寫法可能會感覺很奇怪，以下我們將介紹 lambda expression 的使用方式與時機，並提供幾個範例作為參考。

Lambda expression 是一種匿名函數的表示方式，它可以讓程式設計師將函數的內容直接以 inline 的方式寫在一般的程式碼之中，省去另外定義函數的

麻煩，使用時機跟 functor 與 function pointer 類似，一般的狀況都是使用 lambda expression 定義一個匿名的函數，然後再將此函數當作另外一個函數的傳入參數來使用。

支援 Lambda Expression 的 C++ 編譯器

要開始學習 lambda expression 之前，要先準備支援 lambda expression 的編譯器，由於 lambda expression 是在 C++ 11 才加入的新語法，所以不見得每一種編譯器都有支援，以下這些是有支援 lambda expression 的編譯器版本：

- GCC 4.5：需要指定 `-std=c++11` 參數。
- Intel C++ Compiler 11：需要指定 `/Qstd=c++0x` 參數。
- Microsoft Visual C++ 2010（包含在 Visual Studio 2010 之中）

Lambda Expression 的語法

Lambda expression 基本的用法如下：

```
[=] (int x) mutable throw() -> int
{
    // 函數內容
    int n = x + y;
    return n;
}
```

`[=]`：lambda-introducer，也稱為 capture clause。所有的 lambda expression 都是以它來作為開頭，不可以省略，它除了用來作為 lambda expression 開頭的關鍵字之外，也有抓取（capture）變數的功能，指定如何將目前 scope 範圍之變數抓取至 lambda expression 中使用，而抓取

的方式則分為傳值 (by value) 與傳參考 (by reference) 兩種，跟一般函數參數的傳入方式類似，不過其語法有些不同，以下我們以範例解釋：

- `[]`：只有兩個中括號，完全不抓取外部的變數。
- `[=]`：所有的變數都以傳值 (by value) 的方式抓取。
- `[&]`：所有的變數都以傳參考 (by reference) 的方式抓取。
- `[x, &y]`：x 變數使用傳值、y 變數使用傳參考。
- `[=, &y]`：除了 y 變數使用傳參考之外，其餘的變數皆使用傳值的方式。
- `[&, x]`：除了 x 變數使用傳值之外，其餘的變數皆使用傳參考的方式。

這裡要注意一點，預設的抓取選項 (capture-default，亦即 `=` 或是 `&`) 要放在所有的項目之前，也就是放在第一個位置。

`(int x)`：lambda declarator，也稱為參數清單 (parameter list)。定義此匿名函數的傳入參數列表，基本的用法跟一般函數的傳入參數列表一樣，不過多了一些限制條件：

- 不可指定參數的預設值。
- 不可使用可變長度的參數列表。
- 參數列表不可以包含沒有命名的參數。

參數清單在 lambda expression 中並不是一個必要的項目，如果不需要傳入任何參數的話，可以連同小括號都一起省略。

`mutable`：mutable specification。加入此關鍵字可以讓 lambda expression 直接修改以傳值方式抓取進來的外部變數，若不需要此功能，則可以將其省略。

`throw()`：例外狀況規格 (exception specification)。指定該函數會丟出的例外，其使用的方法跟一般函數的例外指定方式相同。如果該函數沒有丟出例外的功能，則可以直接省略掉。

-> `int` : 傳回值型別 (return type)。指定 lambda expression 傳回值的型別，這個範例是指定傳回值型別為整數 (`int`)，其他的型別則以此類推。如果 lambda expression 所定義的函數很單純，只有包含一個傳回陳述式 (statement) 或是根本沒有傳回值的話，這部分就可以直接省略，讓編譯器自行判斷傳回值的型別。

`mutable` : compound-statement，亦稱為 Lambda 主體 (lambda body)。這個就是匿名函數的內容，就跟一般的函數內容一樣。

Lambda Expression 的範例

透過一些範例可以讓我們更容易了解 lambda expression 的使用時機與其優勢所在。

Hello World

這是一個最簡單的 Hello World 範例。

```
#include <iostream>
using namespace std;
int main() {
    auto lambda = []() { cout << "Hello, Lambda" << endl; };
    lambda();
}
```

由於這裡的 lambda expression 並沒有需要傳入任何參數，所以可以連同小括號一起省略，改寫成這樣：

```
auto lambda = [] { cout << "Hello, Lambda" << endl; };
```

也可以在參數列加上 `void`，明確標示沒有傳入參數：

```
auto lambda = [](void) { cout << "Hello, Lambda" << endl; };
```

或是將傳回值的類型設為 `void`，明確標示這個函數沒有傳回值：

```
auto lambda = [](void) -> void { cout << "Hello, Lambda" << endl; };
```

直接呼叫 Lambda Expression

這個例子是直接呼叫 lambda expression 所定義的匿名函數，將兩個參數傳入其中進行運算，最後再將運算結果傳回來：

```
#include <iostream>
int main() {
    using namespace std;
    int n = [] (int x, int y) { return x + y; }(5, 4);
    cout << n << endl;
}
```

這個程式執行後的輸出為

9

配合 STL 使用

C++ 標準程式庫中有許多的函數在使用時會需要其他的函數作為傳入參數，最常見的就是一些對於陣列的處理函數，這個例子是 `std::count_if` 最簡單的使用方式：

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
// 提供給 std::count_if 用的函數
bool condition(int value) {
    return (value > 5);
}
int main() {
    vector<int> numbers { 1, 2, 3, 4, 5, 10, 15, 20, 25, 35, 45, 50 };
    // 看看 numbers 中有幾個元素符合 condition 函數所定義的判斷條件
    auto count = count_if(numbers.begin(), numbers.end(), condition);
    cout << "Count: " << count << endl;
}
```

這裡我們定義一個 `condition` 函數，作為 `std::count_if` 在判斷時的依據，`std::count_if` 會將每個元素一一傳入 `condition` 函數中檢查，最後傳回所有符合條件的元素個數。

由於 `std::count_if` 所使用到的判斷函數都需要另外定義，這樣會讓程式碼顯得很冗長，我們可以使用 lambda expression 改寫一下，讓整個程式碼更簡潔：

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int main() {
    vector<int> numbers { 1, 2, 3, 4, 5, 10, 15, 20, 25, 35, 45, 50 };
    // 使用 lambda expression 替代原有的 condition 函數
    auto count = count_if(numbers.begin(), numbers.end(),
        [](int x) { return (x > 5); });
    cout << "Count: " << count << endl;
}
```

我們將原本 `condition` 函數所在的位置，直接使用一個 lambda expression 替換，至於傳入的參數與傳回值的類型則維持不變（傳入 `int`，傳回 `bool`）。

參考資料

- [stackoverflow](#)
- [Cprogramming](#)
- [Heresy's Space](#)

C/C++

« 上一頁

下一頁 »

[台南素食] 王品舒果，新米蘭蔬
食，適合素食者用餐

Google 洋灑新願電子春聯，分享
自己創作的春聯給好友

